feedback.py

I'm fixing a terminal Wordle game. I'm uploading feedback.py with this message.

Bug: duplicate letters are scored wrong. With target "grape", guessing "geese" returns
green yellow yellow gray green, but it should return green gray gray gray green.

Task: rewrite only evaluate(target, guess) so that:

1. Exact matches (green) are resolved first.
2. Each letter in the target can satisfy at most one letter in the guess.
3. Remaining letters are yellow only if unused copies are left, otherwise gray.
4. It works for any word length (4, 5, or 6).

Keep the same function name, inputs, and return format (a list of "green"/"yellow"/"gray").
Don't change any other file. Explain the fix in simple terms and give 4 test cases with expected output.

15/10/2026, 18:44:55

Verifying the fix against several tricky duplicate-letter cases.

Verifying the fix against several tricky duplicate-letter cases.

Done

Here's the replacement for `evaluate` (I dropped the leftover half-comment above it, since it described the old bug):

```
def evaluate(target, guess):
    result = ["gray"] * len(guess)
```

```
    # Pass 1: greens
    for i in range(len(guess)):
        if guess[i] == target[i]:
            result[i] = "green"

    # Count target letters not used up by greens
    remaining = {}
    for i in range(len(target)):
        if result[i] != "green":
            remaining[target[i]] = remaining.get(target[i], 0) + 1

    # Pass 2: yellows, using up one copy each time
    for i in range(len(guess)):
        if result[i] == "green":
            continue
        if remaining.get(guess[i], 0) > 0:
            result[i] = "yellow"
            remaining[guess[i]] -= 1

    return result
```

What was wrong: the old code only asked "is this letter anywhere in the target?", so one `e` in "grape" could make every `e` in the guess yellow.

The fix, in plain terms:

1. Mark all exact matches green first.
2. Count the target letters that greens haven't used up. Think of them as a pile of tokens.
3. Go through the remaining guess letters left to right. If a token for that letter is left, the letter is yellow and a token is spent. If not, it stays gray.

Test cases:

Target	Guess	Expected
grape	geese	green, gray, gray, gray, green

Target	Guess	Expected
apple	paper	yellow, yellow, green, yellow, gray
ball	lull	gray, gray, green, green
banana	aaaaaa	gray, green, gray, green, gray, green

Quick check on the first one: the last `e` is green and uses up grape's only `e`, so the other two `e`s have no token left and go gray. The `s` isn't in the word at all.

15/10/2026, 18:45:09

game.py

main.py

words.py

Task 1 is done. Now Task 2. I'm uploading game.py, main.py and words.py.

Change only what's needed so that:

1. Winning shows a clean win message and ends the game.
2. Losing (after 6 accepted guesses) shows a loss message and reveals the target word.
3. Only 6 accepted guesses are allowed.
4. Invalid guesses (wrong length, not letters, not in the word list) show an error
   and do NOT use up a turn.
5. Typing "quit" exits cleanly.

Keep it modular, keep all state in memory, and don't add any new libraries or files.
Explain what you changed in simple terms.

15/10/2026, 18:47:35

Debugging turn-counting logic in a game script.

Debugging turn-counting logic in a game script.

Done

Only `game.py` needs to change. `main.py` and `words.py` stay as they are.

```
import random
from words import WORDS
from feedback import evaluate
```

```
MAX_GUESSES = 6


class WordleGame:
    def __init__(self, length=5):
        self.length = length
        self.words = [w for w in WORDS if len(w) == length]
        self.target = random.choice(self.words)
        self.history = []

    def validate(self, guess):
        """Return an error message, or None if the guess is OK."""
        if len(guess) != self.length:
            return f"Guess must be {self.length} letters long."
        if not guess.isalpha():
            return "Guess must contain letters only."
        if guess not in self.words:
            return "Not in the word list."
        return None

    def run(self):
        print(f"Wordle - {self.length} letters, {MAX_GUESSES}
guesses. Type 'quit' to exit.")
        while len(self.history) < MAX_GUESSES:
            guess = input("> ").strip().lower()
            if guess == "quit":
                print("Thanks for playing!")
                return
            error = self.validate(guess)
            if error:
                print(error)
                continue
            feedback = evaluate(self.target, guess)
            self.history.append((guess, feedback))
            print(" ".join(feedback))
```

```
            if guess == self.target:
                print(f"You win! Solved in
{len(self.history)}/{MAX_GUESSES} guesses.")
                return
        print(f"You lose! The word was: {self.target}")
```

What changed, in plain terms:

1. **Turns now only count accepted guesses.** The old `for _ in range(6)` used up a turn on every loop, even when the guess was invalid. It's now a `while` loop that runs until `self.history` has 6 entries, and only valid guesses get added to `history`. That fixes requirements 3 and 4.
2. **New `validate()` method.** It checks length, letters-only, and "is it in the word list", and returns an error message or `None`. Invalid input prints the error and loops again without using a turn.
3. **Win and loss messages.** A win prints "You win!" with the guess count and ends the game. After 6 accepted guesses, the loop ends and prints "You lose!" with the target word.
4. **Quit is now `"quit"`** (it was `"q"`). It prints a goodbye and returns, so the program exits normally.
5. **Small supporting changes.** `MAX_GUESSES = 6` replaces the hardcoded number, and the filtered word list is saved once as `self.words`, so `validate()` and the target pick use the same list.

`words.py` only has 5-letter words right now, so `WordleGame(4)` or `WordleGame(6)` would crash on `random.choice` of an empty list. Add 4- and 6-letter words to `words.py` before using those lengths.

15/10/2026, 18:47:46



15/10/2026, 18:48:28

That's working as intended. Both invalid guesses showed an error, and the prompt came straight back without using a turn (still 6 guesses left).

To finish checking the rest, try these in the same game:

1. **Not letters:** type `ab3de`. Expected: "Guess must contain letters only."
2. **Valid guess:** type `crane`. Expected: a row of green/yellow/gray words.
3. **Quit:** type `quit`. Expected: "Thanks for playing!" and you're back at the command prompt.
4. **Win:** to test it quickly, temporarily add `print(self.target)` at the end of `__init__` in `game.py`, run again, and type the printed word. Expected: "You win! Solved in 1/6 guesses." Remove that line afterward.
5. **Loss:** run again and enter 6 valid words that aren't the target (for example `apple`, `array`, `eerie`, `crane`, `grape`, `stare`, skipping whichever one is the target). Expected: "You lose! The word was: ..." with the target revealed.

If any of these behave differently, paste the output here and I'll look at it.

15/10/2026, 18:48:31

game.py

main.py

words.py

Task 2 is done and tested. Now Task 3. I'm uploading the current main.py, game.py and words.py.

Add 4-, 5- and 6-letter modes:

1. At the start, ask the player to choose a word length: 4, 5 or 6. Reject anything else and ask again.
2. Pick the target only from the words in words.py that match the chosen length. Don't add new words. If a length has no words, tell me instead of inventing any.
3. Guess validation, the start message and the feedback must all use the chosen length (no hardcoded 5).
4. Everything from Task 1 and 2 must keep working: duplicate-letter feedback, 6-guess limit, invalid guesses not using a turn, win/loss messages, quit.

Keep it modular, keep all state in memory, no new libraries or files.
Explain what you changed in simple terms and list the test cases I should run.

15/10/2026, 18:49:28

Designing the length prompt and error handling flow.

Designing the length prompt and error handling flow.

Done

Two files change: `main.py` and `game.py`. `words.py` and `feedback.py` stay as they are.

main.py

```
from game import WordleGame

VALID_LENGTHS = ("4", "5", "6")


def choose_length():
    """Ask until the player picks 4, 5 or 6. Returns an int, or None
for quit."""
    while True:
        choice = input("Choose word length (4, 5 or 6):
").strip().lower()
        if choice == "quit":
            return None
        if choice in VALID_LENGTHS:
            return int(choice)
        print("Please enter 4, 5 or 6.")


def main():
    while True:
        length = choose_length()
        if length is None:
            print("Thanks for playing!")
            return
        try:
            game = WordleGame(length)
        except ValueError as error:
            print(error)
            continue
        game.run()
        return
```

```
if __name__ == "__main__":
    main()
```

game.py: only `__init__` changes. Everything else stays exactly as it is.

```
    def __init__(self, length=5):
        self.length = length
        self.words = [w for w in WORDS if len(w) == length]
        if not self.words:
            raise ValueError(f"No {length}-letter words in words.py.
Add some to play this mode.")
        self.target = random.choice(self.words)
        self.history = []
```

What changed, in plain terms:

1. **Length prompt.** `main.py` now asks for 4, 5 or 6 and keeps asking until you give one of those. Typing `quit` there exits cleanly too.
2. **Words come only from `words.py`.** `WordleGame` already filtered the list by length. Now, if nothing matches, it raises an error with a clear message instead of crashing on `random.choice`. `main.py` prints that message and asks for a length again.
3. **Nothing is hardcoded to 5.** The start message, the length check in `validate()` and the word-list check all use `self.length`. `evaluate()` already uses the guess's own length.

Heads-up: `words.py` only has 5-letter words, so picking 4 or 6 will show the "No 4-letter words..." message. That is the behavior you asked for. Add your own 4- and 6-letter words to `words.py` when you want those modes playable.

Test cases:

#	What to do	Expected
1	Run, type `7`, `abc`, `5.0`, or press Enter	"Please enter 4, 5 or 6." each time, then asks again

#	What to do	Expected
2	Type `4`, then `6`	"No 4-letter words in words.py..." / same for 6, then it asks for a length again
3	Type `quit` at the length prompt	"Thanks for playing!" and back to the command prompt
4	Type `5`	"Wordle - 5 letters, 6 guesses. Type 'quit' to exit."
5	In a 5 game, enter `cat`, `ab3de`, `zzzzz`	The three errors from Task 2, and the guess counter doesn't move
6	Enter 6 valid words that aren't the target	"You lose! The word was: ..."
7	Guess the target	"You win! Solved in N/6 guesses."
8	Type `quit` mid-game	"Thanks for playing!" and exit

To check duplicate-letter feedback and the 4- and 6-letter logic without adding words, run this in a Python shell from the project folder:

```
from feedback import evaluate
print(evaluate("grape", "geese"))   # green, gray, gray, gray, green
print(evaluate("ball", "lull"))     # gray, gray, green, green
print(evaluate("banana", "aaaaaa")) # gray, green, gray, green, gray, green
```

15/10/2026, 18:49:42

Checking words.py, it only contains 5-letter words, so 4- and 6-letter modes can't work.
Please update words.py to add 15 common English words each for 4-letter and 6-letter modes,
keeping the same data structure and style as the existing list.
All words must be lowercase,
letters only, and exactly the right length. Include a few with repeated letters
(e.g. "moon", "balloon" is too long, so for 6 letters use words like "coffee", "bottle").
Don't change any other file unless the new words require it, and keep everything in memory.
Show me the full updated words.py.

15/10/2026, 18:51:24

Building a word list with set lengths and line layout.

Building a word list with set lengths and line layout.

Done

```
WORDS = [
    "apple", "array", "eerie", "crane", "grape",
    "stare", "steam", "stone", "plant", "flame",
    "moon", "book", "door", "ball", "seed",
    "keep", "tree", "fish", "bird", "cake",
    "lamp", "road", "wind", "rain", "gold",
    "coffee", "bottle", "button", "summer", "winter",
    "garden", "planet", "pocket", "yellow", "little",
    "sunset", "rabbit", "market", "orange", "bridge",
]
```

- **4-letter (15):** moon, book, door, ball, seed, keep, tree, fish, bird, cake, lamp, road, wind, rain, gold

- **6-letter (15):** coffee, bottle, button, summer, winter, garden, planet, pocket, yellow, little, sunset, rabbit, market, orange, bridge
- **Repeated letters:** moon, book, door, ball, seed, keep and tree (4-letter), and coffee, bottle, button, summer, pocket, yellow, little and rabbit (6-letter).

It's still one `WORDS` list, so no other file needs to change. `game.py` already picks words by length.

Quick test:

1. Run `python main.py`, type `4`, and the start message should say "Wordle - 4 letters, 6 guesses."
2. Guess `moon`, `book` or any other 4-letter word from the list. Words outside the list should still give "Not in the word list."
3. Repeat with `6`, using `coffee` or `bottle`.

15/10/2026, 18:51:31

feedback.py

game.py

main.py

Task 3 is done and tested. Now Task 4. I'm uploading the current main.py, game.py and feedback.py.

Add guess history and a session summary:

1. After each ACCEPTED guess, print a readable history of all accepted guesses so far,
   numbered, showing each guess and its feedback (e.g. "1. SPEED gray yellow yellow gray gray").
2. Invalid guesses (wrong length, not in word list) must NOT appear in the history or use a turn.
3. At the end of the game (win, loss or quit), print a summary: word length, target word,
   result (won/lost/quit), number of accepted guesses, and the full history.
4. Everything from Tasks 1-3 must keep working.

Keep it modular (put history/summary code in its own functions), keep all state in memory,
no new libraries or files. Explain what you changed in simple terms.

15/10/2026, 18:53:36

Designing a clear, step-by-step game.py update.

Designing a clear, step-by-step game.py update.

Done

Only `game.py` changes. `main.py` and `feedback.py` stay as they are.

```python
import random
from words import WORDS
from feedback import evaluate

MAX_GUESSES = 6


def format_history(history):
    """Turn [(guess, feedback), ...] into numbered lines like '1. SPEED  gray yellow ...'."""
    return [
        f"{n}. {guess.upper()}  {' '.join(feedback)}"
        for n, (guess, feedback) in enumerate(history, start=1)
    ]


def print_history(history):
    print("--- Guesses so far ---")
    for line in format_history(history):
        print(line)


def print_summary(length, target, result, history):
    print()
    print("===== Game summary =====")
    print(f"Word length: {length}")
    print(f"Target word: {target.upper()}")
    print(f"Result: {result}")
    print(f"Accepted guesses: {len(history)}/{MAX_GUESSES}")
    print("History:")
    if history:
        for line in format_history(history):
            print(line)
    else:
        print("(no guesses)")
```

```
class WordleGame:
    def __init__(self, length=5):
        self.length = length
        self.words = [w for w in WORDS if len(w) == length]
        if not self.words:
            raise ValueError(f"No {length}-letter words in words.py.
Add some to play this mode.")
        self.target = random.choice(self.words)
        self.history = []

    def validate(self, guess):
        """Return an error message, or None if the guess is OK."""
        if len(guess) != self.length:
            return f"Guess must be {self.length} letters long."
        if not guess.isalpha():
            return "Guess must contain letters only."
        if guess not in self.words:
            return "Not in the word list."
        return None

    def run(self):
        print(f"Wordle - {self.length} letters, {MAX_GUESSES}
guesses. Type 'quit' to exit.")
        result = "lost"
        while len(self.history) < MAX_GUESSES:
            guess = input("> ").strip().lower()
            if guess == "quit":
                result = "quit"
                break
            error = self.validate(guess)
            if error:
                print(error)
                continue
```

```
            feedback = evaluate(self.target, guess)
            self.history.append((guess, feedback))
            print_history(self.history)
            if guess == self.target:
                result = "won"
                break

        if result == "won":
            print(f"You win! Solved in
{len(self.history)}/{MAX_GUESSES} guesses.")
        elif result == "lost":
            print(f"You lose! The word was: {self.target}")
        else:
            print("Thanks for playing!")
        print_summary(self.length, self.target, result, self.history)
```

What changed, in plain terms:

1. **Three new helper functions** at the top of the file: `format_history` builds the numbered lines, `print_history` shows them, and `print_summary` prints the end-of-game report.
2. **History after every accepted guess.** The old single line of feedback is replaced by the full numbered list, since the newest guess is already the last line of it.
3. `run()` **now tracks a** `result` (`won`, `lost` or `quit`) and leaves the loop with `break` instead of `return`. That way all three endings pass through the same summary code at the bottom.
4. **Invalid guesses are untouched.** They still hit `continue` before anything is added to `history`, so they never appear in it or use a turn.

Test cases:

#	What to do	Expected
1	Enter `cat`, then `zzzzz` (5 mode)	Errors only. No history printed, counter unchanged

#	What to do	Expected
2	Enter `crane`	History shows `1. CRANE ...` with 5 colors
3	Enter a second valid word	History shows lines 1 and 2, in order
4	Guess the target	Full history, "You win! Solved in N/6", then the summary with Result: won
5	Enter 6 valid wrong words	"You lose! The word was: ...", then the summary with Result: lost, 6/6
6	Type `quit` after 2 guesses	"Thanks for playing!", then the summary with Result: quit, 2/6 and 2 history lines
7	Type `quit` before any guess	Summary with "(no guesses)"
8	Play in 4 or 6 mode	Same behavior, and the summary shows the right word length

Typing `quit` at the "Choose word length" prompt still exits without a summary, because no game has started yet.

15/10/2026, 18:53:50